



Documentação Funcional — AquaSchedule

1. Objetivo do Sistema

O **AquaSchedule** é um sistema de agendamento de aulas de natação que permite a **professores** criarem e gerenciarem turmas com horários fixos, e a **alunos** realizarem reservas nessas turmas.

O sistema garante:

- **Controle de capacidade máxima** por turma, impedindo overbooking.
- **Prevenção de conflitos de horário** para os alunos, utilizando um algoritmo de sobreposição de intervalos.
- **Segregação de acesso** entre os perfis Professor e Aluno, garantindo que cada tipo de usuário execute apenas as ações permitidas ao seu papel.

2. Regras de Negócio

♦ RF001 — Controle de Capacidade

O agendamento de uma reserva (POST /api/reservas) deve falhar com **400 Bad Request** se a quantidade de reservas ativas na turma já tiver atingido a capacidade máxima definida.

- **Validação:** Compara a contagem de reservas com Status = Ativa contra o campo CapacidadeMaxima da turma.
- **Mensagem retornada:** "RF001: A turma '{nome}' já atingiu a capacidade máxima de {n} alunos."
- **Onde é validada:** ReservaService.cs (linhas 60-64)

♦ RF002 — Conflito de Horário

O agendamento de uma reserva deve falhar com **400 Bad Request** se o aluno já possuir uma reserva ativa em outra turma cujo intervalo de horário se sobreponha ao da turma solicitada.

- **Validação:** Utiliza o método AlunoHasConflictAsync que verifica sobreposição de intervalos (DataHoraInicio / DataHoraFim) entre a turma desejada e as turmas já reservadas pelo aluno.

- **Algoritmo de sobreposição:** $\text{InicioA} < \text{FimB} \ \&\& \ \text{InicioB} < \text{FimA}$ (se ambas as condições forem verdadeiras, há conflito).
- **Mensagem retornada:** "RF002: Você já possui uma aula agendada que conflita com este horário ({início} - {fim})."
- **Onde é validada:** ReservaService.cs (linhas 71-75) e ReservaRepository.cs (linhas 64-73)

♦ RF003 — Perfil Professor

Usuários com a role **Professor** possuem as seguintes permissões:

Ação	Endpoint	Descrição
Criar turma	POST /api/turmas	Cria uma nova turma. O ProfessorId é extraído automaticamente do token JWT.
Editar turma	PUT /api/turmas/{id}	Atualiza nome, descrição, modalidade, horários e capacidade de uma turma.
Deletar turma	DELETE /api/turmas/{id}	Remove uma turma do sistema.
Listar todas as reservas	GET /api/reservas	Visualiza todas as reservas de todos os alunos do sistema.
Visualizar detalhes	GET /api/reservas/{id}	Acessa os detalhes de qualquer reserva do sistema.

Controle de Acesso: Aplicado através do atributo [Authorize(Roles = "Professor")] nos controllers TurmasController.cs e ReservasController.cs.

♦ RF004 — Perfil Aluno

Usuários com a role **Aluno** possuem as seguintes permissões:

Ação	Endpoint	Descrição
Listar turmas	GET /api/turmas	Visualiza todas as turmas disponíveis com

Ação	Endpoint	Descrição
		vagas em tempo real.
Ver detalhes de turma	GET /api/turmas/{id}	Visualiza detalhes de uma turma específica.
Criar reserva	POST /api/reservas	Agenda uma reserva para si (sujeito às validações RF001 e RF002).
Listar suas reservas	GET /api/reservas	Visualiza apenas suas próprias reservas.
Cancelar reserva	DELETE /api/reservas/{id} }	Cancela (soft delete) apenas suas próprias reservas.

- **Restrições de Segurança:**

- Tentativas de cancelar reservas de outros alunos resultam em **401 Unauthorized** com a mensagem: "Você não tem permissão para cancelar esta reserva."
- Tentativas de acessar detalhes de reservas de outros alunos resultam em **403 Forbidden**.
- Alunos **não podem** criar, editar ou deletar turmas (retorna **403 Forbidden**).

- **Onde é controlado:** ReservaService.cs (linhas 98-101)

- ♦ **RF005 — Unicidade de E-mail**

O sistema não permite o cadastro de dois usuários com o mesmo endereço de e-mail. A unicidade é garantida em **duas camadas**:

- **Camada de aplicação:** O AuthService verifica via EmailExistsAsync antes de criar o usuário, retornando 400 Bad Request com a mensagem: "Este e-mail já está cadastrado."
- **Camada de banco de dados:** Índice único (HasIndex(u => u.Email).IsUnique()) no ApplicationDbContext.
- **Onde é validada:** AuthService.cs (linha 33-35) e ApplicationDbContext.cs (linha 30)

♦ RF006 — Validação de Senha

A senha informada no registro deve possuir **no mínimo 6 caracteres**. Caso contrário, a requisição é rejeitada com 400 Bad Request.

- **Validação:** Atributo [MinLength(6)] no DTO RegisterRequestDTO.
- **Mensagem retornada:** "A senha deve ter no mínimo 6 caracteres."
- **Armazenamento:** A senha é criptografada como hash BCrypt (BCrypt.Net.BCrypt.HashPassword), nunca em texto plano.
- **Onde é validada:** RegisterRequestDTO.cs (linha 16-18)

♦ RF007 — Role Padrão no Registro

Quando o campo Role não é informado no registro, o sistema assume automaticamente o perfil **Aluno** como valor padrão. Somente os valores "Aluno" e "Professor" são aceitos.

- **Validação:** Conversão via Enum.TryParse<UsuarioRole> no AuthService. Valores inválidos retornam 400 Bad Request.
- **Mensagem retornada:** "Role inválida. Use 'Aluno' ou 'Professor'."
- **Onde é validada:** AuthService.cs (linha 37-39) e RegisterRequestDTO.cs (linha 23)

♦ RF008 — Reserva Duplicada na Mesma Turma

O sistema impede que um aluno crie **mais de uma reserva ativa** na mesma turma. Tentativas resultam em 400 Bad Request.

- **Validação:** O método AlunoJaReservouTurmaAsync verifica se já existe uma reserva com Status = Ativa para o par (AlunoId, TurmaId).
- **Mensagem retornada:** "Você já possui uma reserva ativa nesta turma."
- **Onde é validada:** ReservaService.cs (linha 67-69) e ReservaRepository.cs (linha 75-82)

♦ RF009 — Cancelamento por Soft Delete

O cancelamento de uma reserva **não remove o registro do banco de dados**. Em vez disso, o campo Status é alterado de Ativa para Cancelada, preservando o histórico completo de agendamentos.

- **Comportamento:** Reservas canceladas não são contabilizadas no cálculo de vagas disponíveis (RF001) nem na detecção de conflitos de horário (RF002).

- **Restrição adicional:** Tentativas de cancelar uma reserva já cancelada retornam 400 Bad Request com a mensagem: "Esta reserva já foi cancelada."
- **Onde é validada:** ReservaService.cs (linha 103-104)

♦ RF010 — Limite de Capacidade da Turma

A capacidade máxima de alunos por turma deve ser um valor entre **1 e 50**. Valores fora desse intervalo são rejeitados com 400 Bad Request tanto na criação quanto na atualização da turma.

- **Validação:** Atributo [Range(1, 50)] nos DTOs TurmaCreateDTO e TurmaUpdateDTO, e também no Model Turma.
- **Mensagem retornada:** "A capacidade deve ser entre 1 e 50 alunos."
- **Onde é validada:** TurmaCreateDTO.cs (linha 29-31) e Turma.cs (linha 47-48)

♦ RF011 — Identidade Extraída do Token (Segurança por Design)

O sistema **nunca aceita IDs de usuário via corpo (body) da requisição** para operações sensíveis. O AlunoId (em reservas) e o ProfessorId (em turmas) são sempre extraídos do claim NameIdentifier do token JWT autenticado.

- **Motivação:** Impedir que um aluno crie reservas em nome de outro, ou que um professor associe turmas a outro docente.
- **Implementação:** O ReservaCreateDTO contém apenas o campo TurmaId — sem campo AlunoId.
- **Onde é implementada:** ReservasController.cs (linhas 109-113) e TurmasController.cs (linhas 112-116)

♦ RF012 — Integridade Referencial e Exclusão

O banco de dados aplica regras estritas de integridade referencial para prevenir a exclusão acidental de dados relacionados:

Relacionamen to	Comportamento ao Deletar	Justificativa
Professor → Turma	Restrict	Impede deletar um professor que possui turmas vinculadas, protegendo a integridade das aulas.
Aluno → Reserva	Restrict	Impede deletar um aluno que possui reservas ativas ou passadas para

Relacionamen to	Comportamento ao Deletar	Justificativa
		preservar o histórico.
Turma → Reserva	Cascade	Ao deletar uma turma, remove automaticamente suas reservas, pois elas perdem o sentido no domínio.

- **Onde é configurada:** AppDbContext.cs (linhas 36-58)

◆ RF013 — Tratamento Padronizado de Erros

Todas as exceções lançadas pela camada de Service são interceptadas por um middleware global e convertidas em respostas HTTP padronizadas, **sem vazamento de stack traces**:

Tipo de Exceção	Status HTTP	Uso no Sistema
InvalidOperationException	400 Bad Request	Violações de regras de negócio (RF001, RF002, RF008, RF009)
ArgumentException	400 Bad Request	Validação de parâmetros inválidos (RF007)
UnauthorizedAccessException	401 Unauthorized	Credenciais inválidas ou ação não permitida (RF004)
KeyNotFoundException	404 Not Found	Recurso não encontrado (ex: turma inexistente ao reservar)
Qualquer outra exceção	500 Internal Server Error	Erro genérico. Mensagem: "Ocorreu um erro interno no servidor."

- **Formato da resposta:** JSON padronizado com os campos status, message e timestamp (UTC).
- **Onde é implementado:** GlobalExceptionHandlerMiddleware.cs (linhas 44-73)

3. Funcionalidades Principais



Autenticação e Segurança

- **Registro de usuários:** Cadastro com definição de perfil (Aluno ou Professor) e validação de e-mail único.
- **Login com Token JWT:** Geração de token contendo claims de identidade (Id, Email, Nome, Role), com expiração de 24 horas.
- **Segurança de Senhas:** Hash criptográfico feito via BCrypt.
- **Validação do Token:** Verificação rigorosa de Issuer, Audience, Lifetime e assinatura HMAC-SHA256.



Gestão de Turmas

- **CRUD Completo:** Controle total de turmas contendo nome, descrição, modalidade, data/hora de início/fim e capacidade máxima.
- **Cálculo Dinâmico de Vagas:** Retorno em tempo real utilizando a fórmula:
$$VagasDisponiveis = CapacidadeMaxima - ReservasAtivas$$
- **Vínculo Automático:** Atribuição do professor responsável extraída diretamente do token JWT.



Agendamento de Reservas

- **Validação Automática:** Bloqueio de agendamentos por choque de horários (RF002) ou falta de vagas (RF001).
- **Soft Delete no Cancelamento:** O status passa para Cancelada, liberando a vaga da turma e o horário do aluno, mas mantendo a trilha de auditoria no banco.
- **Visão Segregada:** Alunos enxergam apenas o próprio histórico; professores têm visão gerencial de todas as reservas.



Interface e Extras do Sistema

- **Frontend Integrado:** Interface web responsiva baseada em HTML5/JavaScript (ES6+) com chamadas fetch autenticadas, estilizada com **Bootstrap 5.3 (Tema Dark)** e servida via arquivos estáticos (wwwroot).
- **Documentação Interativa:** Swagger UI disponível no endpoint /swagger em ambiente de desenvolvimento, configurado com suporte a autenticação Bearer Token.

- **Testes Automatizados:** Suíte de testes unitários criada com **xUnit e Moq**, validando os cenários críticos